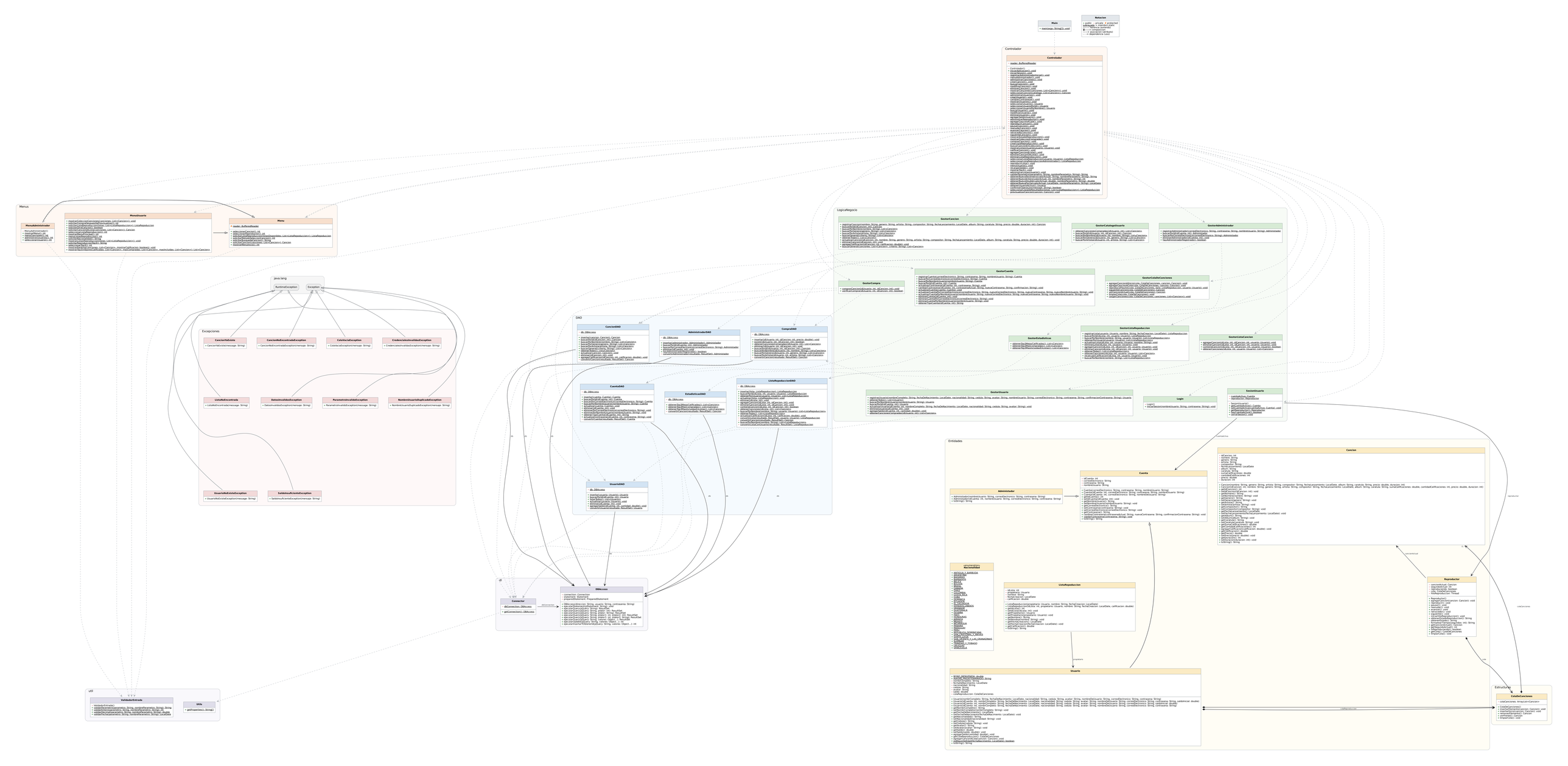

Main
+ main(args: String[]): void
Notacion
+ public - private # protected
subrayado = miembro static
△—— herencia (extends)
◆——> composicion
——> asociacion (atributo)
- - -> dependencia (uso)
Controlador
Controlador
- reader: BufferedReader
- Controlador()
+ iniciarAplicacion(): void
- iniciarSesion(): void
- registrarAdministradorInicial(): void
- menuAdministrador(): void
- administrarCanciones(): void
- crearCancion(): void
- buscarCancion(): void
- modificarCancion(): void
- eliminarCancion(): void
- mostrarCanciones(canciones: List<Cancion>): void
- seleccionarCancion(catalogo: List<Cancion>): Cancion
- administrarUsuarios(): void
- crearUsuario(): void
- cambiarContrasena(): void
- mostrarUsuarios(): void
- seleccionarUsuario(): Usuario
- seleccionarUsuarioPorId(): Usuario
- seleccionarUsuarioPorNombre(): Usuario
- buscarUsuario(): void
- modificarUsuario(): void
- eliminarUsuario(): void
- agregarSaldoUsuario(): void
- administrarReproductor(): void
- agregarCancionACola(): void
- reproducirCancion(): void
- pausarCancion(): void
- reanudarCancion(): void
- avanzarCancion(): void
- retrocederCancion(): void
- siguienteCancion(): void
- mostrarEstadoReproduccion(): void
- mostrarColeccionComprada(): void
- comprarCancion(): void
- crearListaReproduccion(): void
- buscarCancionEnColeccion(): void
- mostrarListasUsuario(usuario: Usuario): void
- calificarCancion(): void
- agregarCancionALista(): void
- eliminarCancionDeLista(): void
- eliminarListaReproduccion(): void
- seleccionarListaReproduccion(usuario: Usuario): ListaRepoduccion
- seleccionarListaReproduccionAdministrador(): ListaRepoduccion
- reproducirLista(): void
- menuUsuario(): void
- recargarSaldo(): void
- mostrarTop3(): void
- administrarListasUsuario(): void
+ validarParametro(parametro: String, nombreParametro: String): String
- obtenerNuevoParametro(valorActual: String, nombreParametro: String): String
- obtenerNuevoEntero(valorActual: int, nombreParametro: String): int
- obtenerNuevoDouble(valorActual: double, nombreParametro: String): double
- obtenerNuevaFecha(valorActual: LocalDate, nombreParametro: String): LocalDate
- obtenerUsuarioActivo(): Usuario
- confirmarOperacion(mensaje: String): boolean
- seleccionarListaDeResultados(listas: List<ListaRepoduccion>): ListaRepoduccion
- previsualizarCancion(cancion: Cancion): void
Menus
MenuAdministrador
- MenuAdministrador()
+ mostrarMenu(): int
+ menuCanciones(): int
+ administrarUsuarios(): int
+ seleccionarUsuario(): int
MenuUsuario
+ mostrarColeccionCanciones(canciones: List<Cancion>): void
+ solicitarCompraDespuesDePrevisualizar(): int
+ solicitarListaReproduccion(listas: List<ListaRepoduccion>): ListaRepoduccion
+ solicitarOtraCancion(): boolean
+ solicitarCancionDeLista(canciones: List<Cancion>): Cancion
+ seleccionarListaReproduccion(): int
+ mostrarMenuPrincipal(): int
+ menuListasReproduccion(): int
+ solicitarNacionalidad(): String
+ mostrarListasReproduccion(listas: List<ListaRepoduccion>): void
+ solicitarReproduccionTop3(): String
+ seleccionarTop3(): int
- mostrarRanking(canciones: List<Cancion>, mostrarCalificacion: boolean): void
+ mostrarTop3(mejoresCalificadas: List<Cancion>, masCompradas: List<Cancion>, masIncluidas: List<Cancion>): List<Cancion>
Menu
# reader: BufferedReader
+ seleccionarCancion(): int
+ seleccionarReproductor(): int
+ solicitarListaReproduccion(listasDisponibles: List<ListaRepoduccion>): ListaRepoduccion
+ solicitarMetodoSeleccionCancion(): int
+ solicitarBusquedaCancion(): String
+ solicitarCancion(canciones: List<Cancion>): Cancion
+ mostrarMenuInicio(): int
java.lang
RuntimeException
Exception
Excepciones
CancionYaExiste
+ CancionYaExiste(message: String)
CancionNoEncontradaException
+ CancionNoEncontradaException(mensaje: String)
ColaVaciaException
+ ColaVaciaException(mensaje: String)
CredencialesInvalidasException
+ CredencialesInvalidasException(mensaje: String)
ListaNoEncontrada
+ ListaNoEncontrada(message: String)
DatosInvalidosException
+ DatosInvalidosException(mensaje: String)
ParametroInvalidoException
+ ParametroInvalidoException(message: String)
NombreUsuarioDuplicadoException
+ NombreUsuarioDuplicadoException(mensaje: String)
UsuarioNoExisteException
+ UsuarioNoExisteException(message: String)
SaldoInsuficienteException
+ SaldoInsuficienteException(mensaje: String)
util
ValidadorEntrada
- ValidadorEntrada()
+ validarParametro(parametro: String, nombreParametro: String): String
+ validarEntero(parametro: String, nombreParametro: String): int
+ validarDecimal(parametro: String, nombreParametro: String): double
+ validarFecha(parametro: String, nombreParametro: String): LocalDate
Utils
+ getProperties(): String[]
DAO
CancionDAO
- db: DBAccess
+ insertar(cancion: Cancion): Cancion
+ buscarPorId(idCancion: int): Cancion
+ buscarPorNombre(nombre: String): List<Cancion>
+ buscarPorGenero(genero: String): List<Cancion>
+ buscarPorArtista(artista: String): List<Cancion>
+ buscarGeneral(criterio: String): List<Cancion>
+ obtenerTodas(): List<Cancion>
+ actualizar(cancion: Cancion): void
+ eliminar(idCancion: int): void
+ agregarCalificacion(idCancion: int, calificacion: double): void
- convertirCancion(resultado: ResultSet): Cancion
AdministradorDAO
- db: DBAccess
+ insertar(administrador: Administador): Administador
+ buscarPorId(idCuenta: int): Administador
+ buscarPorCorreoElectronico(correoElectronico: String): Administador
+ eliminar(idCuenta: int): void
+ hayAdministradorRegistrado(): boolean
- convertirAdministrador(resultado: ResultSet): Administador
CompraDAO
- db: DBAccess
+ insertar(idUsuario: int, idCancion: int, precio: double): void
+ existe(idUsuario: int, idCancion: int): boolean
+ obtenerCancionesCompradas(idUsuario: int): List<Cancion>
+ eliminar(idUsuario: int, idCancion: int): void
+ buscarPorId(idUsuario: int, idCancion: int): Cancion
+ buscarPorNombre(idUsuario: int, nombre: String): List<Cancion>
+ buscarPorGenero(idUsuario: int, genero: String): List<Cancion>
+ buscarPorArtista(idUsuario: int, artista: String): List<Cancion>
- convertirCancion(resultado: ResultSet): Cancion
CuentaDAO
- db: DBAccess
+ insertar(cuenta: Cuenta): Cuenta
+ buscarPorId(idCuenta: int): Cuenta
+ buscarPorCorreoElectronico(correoElectronico: String): Cuenta
+ buscarPorNombreUsuario(nombreUsuario: String): Cuenta
+ actualizar(cuenta: Cuenta): void
+ eliminar(idCuenta: int): void
+ eliminarPorCorreoElectronico(correoElectronico: String): void
+ eliminarPorNombreUsuario(nombreUsuario: String): void
+ obtenerTipoCuenta(idCuenta: int): String
+ actualizarContrasena(idCuenta: int, contrasena: String): void
- convertirCuenta(resultado: ResultSet): Cuenta
EstadisticasDAO
- db: DBAccess
+ obtenerTop3MejorCalificadas(): List<Cancion>
+ obtenerTop3MasCompradas(): List<Cancion>
+ obtenerTop3MasIncluidasEnListas(): List<Cancion>
- convertirCancion(resultado: ResultSet): Cancion
ListaRepoduccionDAO
- db: DBAccess
+ insertar(lista: ListaRepoduccion): ListaRepoduccion
+ buscarPorId(idLista: int, usuario: Usuario): ListaRepoduccion
+ obtenerPorUsuario(usuario: Usuario): List<ListaRepoduccion>
+ actualizar(lista: ListaRepoduccion): void
+ eliminar(idLista: int): void
+ agregarCancion(idLista: int, idCancion: int): void
+ eliminarCancion(idLista: int, idCancion: int): void
+ contieneCancion(idLista: int, idCancion: int): boolean
+ obtenerCanciones(idLista: int): List<Cancion>
+ buscarPorNombre(nombre: String, usuario: Usuario): List<ListaRepoduccion>
+ obtenerTodas(): List<ListaRepoduccion>
+ actualizarCalificacion(idLista: int, calificacion: double): void
- convertirLista(resultado: ResultSet, usuario: Usuario): ListaRepoduccion
- convertirCancion(resultado: ResultSet): Cancion
+ buscarPorNombre(nombre: String): List<ListaRepoduccion>
- convertirListaConUsuario(resultado: ResultSet): ListaRepoduccion
UsuarioDAO
- db: DBAccess
+ insertar(usuario: Usuario): Usuario
+ buscarPorId(idCuenta: int): Usuario
+ listarTodos(): List<Usuario>
+ actualizar(usuario: Usuario): void
+ eliminar(idCuenta: int): void
+ agregarSaldo(idCuenta: int, cantidad: double): void
- convertirUsuario(resultado: ResultSet): Usuario
db
db
db
db
db
db
db
dl
Connector
- dbConnection: DBAccess
+ getConnection(): DBAccess
dbConnection
DBAccess
- connection: Connection
- statement: Statement
- preparedStatement: PreparedStatement
+ DBAccess(direccion: String, usuario: String, contrasenia: String)
+ ejecutarStatement(pStatement: String): void
+ ejecutarQuery(pQuery: String): ResultSet
+ ejecutarQuery(pQuery: String, pValor: int): ResultSet
+ ejecutarQuery(pQuery: String, pValor: String): ResultSet
+ ejecutarQuery(pQuery: String, pValor1: int, pValor2: int): ResultSet
+ ejecutarQuery(pQuery: String, pValor1: int, pValor2: String): ResultSet
+ ejecutarQuery(pQuery: String, valores: Object...): ResultSet
+ ejecutarUpdate(pQuery: String, valores: Object...): int
+ ejecutarInsertarYObtenerId(pQuery: String, valores: Object...): int
LogicaNegocio
GestorCancion
+ registrarCancion(nombre: String, genero: String, artista: String, compositor: String, fechaLanzamiento: LocalDate, album: String, caratula: String, precio: double, duracion: int): Cancion
+ buscarPorId(idCancion: int): Cancion
+ buscarPorNombre(nombre: String): List<Cancion>
+ buscarPorGenero(genero: String): List<Cancion>
+ buscarPorArtista(artista: String): List<Cancion>
+ buscarGeneral(criterio: String): List<Cancion>
+ obtenerTodas(): List<Cancion>
+ actualizarCancion(idCancion: int, nombre: String, genero: String, artista: String, compositor: String, fechaLanzamiento: LocalDate, album: String, caratula: String, precio: double, duracion: int): void
+ eliminarCancion(idCancion: int): void
+ agregarCalificacion(idCancion: int, calificacion: double): void
+ buscarGeneral(canciones: List<Cancion>, criterio: String): List<Cancion>
GestorCatalogoUsuario
+ obtenerCancionesCompradas(idUsuario: int): List<Cancion>
+ buscarPorId(idUsuario: int, idCancion: int): Cancion
+ buscarPorNombre(idUsuario: int, nombre: String): List<Cancion>
+ buscarPorGenero(idUsuario: int, genero: String): List<Cancion>
+ buscarPorArtista(idUsuario: int, artista: String): List<Cancion>
GestorAdministrador
+ registrarAdministrador(correoElectronico: String, contrasena: String, nombreUsuario: String): Administador
+ buscarPorId(idCuenta: int): Administador
+ buscarPorCorreoElectronico(correoElectronico: String): Administador
+ eliminarAdministrador(idCuenta: int): void
+ hayAdministradorRegistrado(): boolean
GestorCompra
+ comprarCancion(idUsuario: int, idCancion: int): void
+ verificarCompra(idUsuario: int, idCancion: int): boolean
GestorCuenta
+ registrarCuenta(correoElectronico: String, contrasena: String, nombreUsuario: String): Cuenta
+ buscarPorCorreoElectronico(correoElectronico: String): Cuenta
+ buscarPorNombreUsuario(nombreUsuario: String): Cuenta
+ buscarPorId(idCuenta: int): Cuenta
+ actualizarContrasena(idCuenta: int, contrasena: String): void
+ cambiarContrasena(idCuenta: int, contrasenaActual: String, nuevaContrasena: String, confirmacion: String): void
+ actualizarCuenta(cuenta: Cuenta): void
+ actualizarCuentaPorCorreoElectronico(correoElectronico: String, nuevoCorreoElectronico: String, nuevaContrasena: String, nuevoNombreUsuario: String): void
+ actualizarCuentaPorNombreUsuario(nombreUsuario: String, nuevoCorreoElectronico: String, nuevaContrasena: String, nuevoNombreUsuario: String): void
+ eliminarCuenta(idCuenta: int): void
+ eliminarCuentaPorCorreoElectronico(correoElectronico: String): void
+ eliminarCuentaPorNombreUsuario(nombreUsuario: String): void
+ obtenerTipoCuenta(idCuenta: int): String
GestorColaDeCanciones
+ agregarCancionAlInicio(cola: ColaDeCanciones, cancion: Cancion): void
+ agregarCancionACola(cola: ColaDeCanciones, cancion: Cancion): void
+ reproducirPlaylist(cola: ColaDeCanciones, lista: ListaRepoduccion, usuario: Usuario): void
+ siguienteCancion(cola: ColaDeCanciones): Cancion
+ verCancionActual(cola: ColaDeCanciones): Cancion
+ limpiarCola(cola: ColaDeCanciones): void
+ cargarCanciones(cola: ColaDeCanciones, canciones: List<Cancion>): void
GestorEstadisticas
+ obtenerTop3MejorCalificadas(): List<Cancion>
+ obtenerTop3MasCompradas(): List<Cancion>
+ obtenerTop3MasIncluidasEnListas(): List<Cancion>
GestorListaRepoduccion
+ registrarLista(usuario: Usuario, nombre: String, fechaCreacion: LocalDate): ListaRepoduccion
+ buscarPorId(idLista: int, usuario: Usuario): ListaRepoduccion
+ buscarPorNombre(nombre: String, usuario: Usuario): List<ListaRepoduccion>
+ obtenerPorUsuario(usuario: Usuario): List<ListaRepoduccion>
+ actualizarLista(idLista: int, usuario: Usuario, nombre: String): void
+ eliminarLista(idLista: int, usuario: Usuario): void
+ agregarCancion(idLista: int, idCancion: int, usuario: Usuario): void
+ eliminarCancion(idLista: int, idCancion: int, usuario: Usuario): void
+ obtenerTodas(): List<ListaRepoduccion>
+ obtenerCanciones(idLista: int, usuario: Usuario): List<Cancion>
+ recalcularCalificacion(idLista: int, usuario: Usuario): void
+ buscarPorNombre(nombre: String): List<ListaRepoduccion>
GestorListaCancion
+ agregarCancion(idLista: int, idCancion: int, usuario: Usuario): void
+ eliminarCancion(idLista: int, idCancion: int, usuario: Usuario): void
+ contieneCancion(idLista: int, idCancion: int, usuario: Usuario): boolean
+ obtenerCanciones(idLista: int, usuario: Usuario): List<Cancion>
GestorUsuario
+ registrarUsuario(nombreCompleto: String, fechaDeNacimiento: LocalDate, nacionalidad: String, cedula: String, avatar: String, nombreUsuario: String, correoElectronico: String, contrasena: String, confirmacionContrasena: String): Usuario
+ obtenerTodos(): List<Usuario>
+ buscarPorNombreUsuario(nombreUsuario: String): Usuario
+ buscarPorId(idCuenta: int): Usuario
+ actualizarUsuario(idCuenta: int, nombreCompleto: String, fechaDeNacimiento: LocalDate, nacionalidad: String, cedula: String, avatar: String): void
+ eliminarUsuario(idCuenta: int): void
+ agregarSaldo(idCuenta: int, cantidad: double): void
+ obtenerCancionesCompradas(idUsuario: int): List<Cancion>
Login
- Login()
+ iniciarSesion(nombreUsuario: String, contrasena: String): void
SesionUsuario
- cuentaActiva: Cuenta
- reproductor: Reproductor
- SesionUsuario()
+ getCuentaActiva(): Cuenta
+ setCuentaActiva(cuentaActiva: Cuenta): void
+ getReproductor(): Reproductor
+ hayCuentaActiva(): boolean
+ cerrarSesion(): void
cuentaActiva
reproductor
Entidades
Administador
+ Administador(nombreUsuario: String, correoElectronico: String, contrasena: String)
+ Administador(idCuenta: int, nombreUsuario: String, correoElectronico: String, contrasena: String)
+ toString(): String
Cuenta
- idCuenta: int
- correoElectronico: String
- contrasena: String
- nombreUsuario: String
+ Cuenta(correoElectronico: String, contrasena: String, nombreUsuario: String)
+ Cuenta(idCuenta: int, correoElectronico: String, contrasena: String, nombreUsuario: String)
+ Cuenta(idCuenta: int, correoElectronico: String, nombreDeUsuario: String)
+ getIdCuenta(): int
+ setIdCuenta(idCuenta: int): void
+ getNombreUsuario(): String
+ setNombreUsuario(nombreUsuario: String): void
+ getCorreoElectronico(): String
+ setContrasena(contrasena: String): void
+ setCorreoElectronico(correoElectronico: String): void
+ getContrasena(): String
+ cambiarContrasena(contrasenaActual: String, nuevaContrasena: String, confirmacionContrasena: String): void
+ validarContrasena(contrasena: String): void
+ toString(): String
Cancion
- idCancion: int
- nombre: String
- genero: String
- artista: String
- compositor: String
- fechaLanzamiento: LocalDate
- album: String
- caratula: String
- sumaCalificaciones: double
- cantidadCalificaciones: int
- precio: double
- duracion: int
+ Cancion(nombre: String, genero: String, artista: String, compositor: String, fechaLanzamiento: LocalDate, album: String, caratula: String, precio: double, duracion: int)
+ Cancion(idCancion: int, nombre: String, genero: String, artista: String, compositor: String, fechaLanzamiento: LocalDate, album: String, caratula: String, sumaCalificaciones: double, cantidadCalificaciones: int, precio: double, duracion: int)
+ getIdCancion(): int
+ setIdCancion(idCancion: int): void
+ getNombre(): String
+ setNombre(nombre: String): void
+ getGenero(): String
+ setGenero(genero: String): void
+ getArtista(): String
+ setArtista(artista: String): void
+ getCompositor(): String
+ setCompositor(compositor: String): void
+ getFechaLanzamiento(): LocalDate
+ setFechaLanzamiento(fechaLanzamiento: LocalDate): void
+ getAlbum(): String
+ setAlbum(album: String): void
+ getCaratula(): String
+ setCaratula(caratula: String): void
+ getSumaCalificaciones(): double
+ getCantidadCalificaciones(): int
+ agregarCalificacion(calificacion: double): void
+ getCalificacion(): double
+ getPrecio(): double
+ setPrecio(precio: double): void
+ getDuracion(): int
+ setDuracion(duracion: int): void
+ toString(): String
0..*
cancionActual
«enumeration»
Nacionalidad
+ ANTIGUA_Y_BARBUDA
+ ARGENTINA
+ BAHAMAS
+ BARBADOS
+ BELICE
+ BOLIVIA
+ BRASIL
+ CANADA
+ CHILE
+ COLOMBIA
+ COSTA_RICA
+ CUBA
+ DOMINICA
+ ECUADOR
+ EL_SALVADOR
+ ESTADOS_UNIDOS
+ GRANADA
+ GUATEMALA
+ GUYANA
+ HAITI
+ HONDURAS
+ JAMAICA
+ MEXICO
+ NICARAGUA
+ PANAMA
+ PARAGUAY
+ PERU
+ REPUBLICA_DOMINICANA
+ SAN_CRISTOBAL_Y_NIEVES
+ SANTA_LUCIA
+ SAN_VICENTE_Y_LAS_GRANADINAS
+ SURINAM
+ TRINIDAD_Y_TOBAGO
+ URUGUAY
+ VENEZUELA
ListaRepoduccion
- idLista: int
- propietario: Usuario
- nombre: String
- fechaCreacion: LocalDate
- calificacion: double
+ ListaRepoduccion(propietario: Usuario, nombre: String, fechaCreacion: LocalDate)
+ ListaRepoduccion(idLista: int, propietario: Usuario, nombre: String, fechaCreacion: LocalDate, calificacion: double)
+ getIdLista(): int
+ setIdLista(idLista: int): void
+ getPropietario(): Usuario
+ setPropietario(propietario: Usuario): void
+ getNombre(): String
+ setNombre(nombre: String): void
+ getFechaCreacion(): LocalDate
+ setFechaCreacion(fechaCreacion: LocalDate): void
+ getCalificacion(): double
+ toString(): String
propietario
Reproductor
- cancionActual: Cancion
- segundoActual: int
- reproduciendo: boolean
- cola: ColaDeCanciones
- hiloReproduccion: Thread
+ Reproductor()
+ agregarCancion(cancion: Cancion): void
+ reproducir(): void
+ pausar(): void
+ reanudar(): void
+ avanzar(): void
+ retroceder(): void
+ siguiente(): void
- iniciarHiloReproduccion(): void
+ obtenerEstadoReproduccion(): String
- obtenerEstado(): String
- formatearTiempo(segundos: int): String
+ getCancionActual(): Cancion
+ getSegundoActual(): int
+ isReproduciendo(): boolean
+ getCola(): ColaDeCanciones
+ limpiarCola(): void
colaCanciones
cola
Usuario
+ BONO_BIENVENIDA: double
+ AVATAR_PREDETERMINADO: String
- nombrCompleto: String
- fechaDeNacimiento: LocalDate
- nacionalidad: String
- cedula: String
- avatar: String
- saldo: double
- colaReproduccion: ColaDeCanciones
+ Usuario(nombrCompleto: String, fechaDeNacimiento: LocalDate, nacionalidad: String, cedula: String, avatar: String, nombreDeUsuario: String, correoElectronico: String, contrasena: String)
+ Usuario(idCuenta: int, nombrCompleto: String, fechaDeNacimiento: LocalDate, nacionalidad: String, cedula: String, avatar: String, nombreDeUsuario: String, correoElectronico: String, contrasena: String, saldoInicial: double)
+ Usuario(idCuenta: int, nombrCompleto: String, fechaDeNacimiento: LocalDate, nacionalidad: String, cedula: String, avatar: String, nombreDeUsuario: String, correoElectronico: String, saldoInicial: double)
+ Usuario(idCuenta: int, nombrCompleto: String, fechaDeNacimiento: LocalDate, nacionalidad: String, cedula: String, avatar: String, nombreDeUsuario: String, correoElectronico: String, contrasena: String)
+ getNombrCompleto(): String
+ setNombrCompleto(nombrCompleto: String): void
+ getFechaDeNacimiento(): LocalDate
+ setFechaDeNacimiento(fechaDeNacimiento: LocalDate): void
+ getNacionalidad(): String
+ setNacionalidad(nacionalidad: String): void
+ getCedula(): String
+ setCedula(cedula: String): void
+ getAvatar(): String
+ setAvatar(avatar: String): void
+ getSaldo(): double
+ setSaldo(saldo: double): void
+ agregarSaldo(cantidad: double): void
+ getColaReproduccion(): ColaDeCanciones
+ agregarCancionACola(cancion: Cancion): void
+ esMayorDeEdad(fechaNacimiento: LocalDate): boolean
+ toString(): String
colaReproduccion
Estructuras
ColaDeCanciones
- colaCanciones: ArrayList<Cancion>
+ ColaDeCanciones()
+ insertarElemento(cancion: Cancion): void
+ insertarInicio(cancion: Cancion): void
+ removerElemento(): Cancion
+ verFrente(): Cancion
+ limpiarCola(): void